

A Reservation-based List Scheduling for Embedded Systems with Memory Constraints

Kai-Siang Wang¹ and Jerry Chou²

¹ National Tsing Hua University, Hsinchu, Taiwan kswang@lsalab.cs.nthu.edu.tw

² National Tsing Hua University, Hsinchu, Taiwan jchou@lsalab.cs.nthu.edu.tw

Abstract. Many embedded systems have hard resource constraints that make schedules found by list scheduling heuristics infeasible. One of the main challenges yielded by memory constraints and the high degree of parallelism is deadlock. In this paper, our primary goal is to find a feasible solution given the memory constraints. We propose a reservation-based solution, an extension for list scheduling algorithms, that can be integrated into those algorithms and make them aware of memory constraints. We show our technique prevents deadlock and significantly reduces the required memory size. The experimental results on randomly generated graphs and real world applications show that our proposed solution can obtain relatively high-quality solutions with up to 10% makespan improvement and 30% memory reduction on average.

Keywords: DAG scheduling · Memory Constraints · Optimization.

1 Introduction

Task scheduling problem has been presented for a long time and is usually formatted as a DAG(Directed Acyclic Graph). Since it has been proved to be NP-hard, many scheduling heuristics have been proposed, and variations based on them have emerged. They can be formulated into three categories: List Scheduling Heuristics, Clustering Heuristics, and Task Duplication Heuristics. Among them, list scheduling is relatively common, and because of its effectiveness, many applications have adopted this group of algorithms.

List scheduling usually has two phases. The first phase consists of calculating the priority of each task in a given metric, the average bottom level in HEFT [11], for example. After that, we sort the tasks with their priorities in descending order. Then, in the second phase, we follow the order of the list and assign each task to the processor with another predefined metric. i.e., EFT(Earliest Finish Time) in HEFT. At the end of this process, a static schedule is generated as illustrated with Algorithm 1.

Traditionally, there are a few limitations in real-world applications, such as network delay and heterogeneity cause people to make some modifications. For example, HEFT [11] takes the communication costs and different computation costs based on each processor into account. In multi-processor systems or cloud computing, such scheduling algorithm has greatly succeeded.

Nowadays, however, deep learning applications have emerged drastically. Those applications adopt one or multiple huge DNN models to help people solve significant tasks, e.g., Object-detection and NLP. However, although GPUs often carry out model training, given their massive parallelism, simple control flow, and high energy efficiency, model inference in the market includes FPGA and ASIC chips. Since those devices often have scarce memory resources, scheduling in embedded devices could be challenging and is not a trivial problem. The scheduling algorithms that do not take memory into account may lead to poor schedules. Furthermore, a schedule could occur **deadlocks** if it's not aware of **memory constraints**.

In this paper, we propose a reservation-based solution that addresses the issues caused by memory constraints. This solution can be integrated into most list scheduling algorithms. We validate our solutions in terms of effectiveness with two examples: HEFT and CPOP.

2 Related Work

2.1 Scheduling with Shared Memory Constraints

One way to improve general list scheduling algorithms such as HEFT is to adjust their priority functions. Since in HEFT, priorities are only decided by the rank of each task, it may inevitably fail to address the problem caused by additional requirements such as memory constraints. This applies to most list scheduling algorithms whose objectives are to minimize makespan only. Due to the aggressiveness of finding shorter makespan, they usually require more memory and thus fail to conform to memory constraints. Therefore, in [1], the main goal of their priority adjustment is to find a priority list in which no two sets of tasks in different memory clusters overlap. This action prevents the algorithm from finding the schedule that uses multiple memory sets simultaneously. Based on the original priorities, they give priority bonuses to the tasks that follow the conditions they define. However, although the priority adjustment may lead to a good priority list, the processor selection phase could still cause deadlocks. In addition, it is more suitable for computation-bound workflows where it can partition the graph easier. We show that when the workflows are memory bound, the case in our definitions, this method might perform worse than *delaying*, a naive solution we provide in Section 5.

2.2 Analysis

There are several works [2, 3, 9] that discuss what is the maximal memory size required given a DAG. When scheduling algorithms are scheduling with the memory of maximal size, they can eventually find a feasible solution. They all use the max-cut algorithm to find maximum memory usage given any timestamp to find the maximum memory size. Once they find the maximum size, they can promise any workflow to be free from deadlock. Furthermore, they even consider

the number of processors to make this bound more accurate. Although we know the boundary it provides, in real-world applications, the memory usage is usually less than that bound due to the schedule found by scheduling algorithms.

3 Challenges: Scheduling with memory constraints

3.1 Memory constraints

Generally, if the memory required by the tasks in a DAG is considerable enough, memory size is one of the challenges of scheduling the graph on embedded devices. For example, deploying a large DNN model on embedded devices. Embedded devices usually have smaller memory than clustering servers or even regular PCs. The objective of most traditional scheduling algorithms is to find the minimized makespan. Since their goals are the minimized execution time and they are unaware of memory constraints, they may inevitably sacrifice the optimization opportunity toward memory usage. As a result, this prevents them from finding better solutions or even feasible solutions under memory size limitations. For this purpose, it is crucial to design a specific optimization method to reduce the execution time as much as possible and conform to the given memory constraint. Many previous works [6, 10] have already taken memory as their primary objective. For instance, Abraham [10] adopts a graph coloring algorithm with an overlapping technique to minimize total memory usage. In this situation, coloring can impact the schedule since two tasks with the same color cannot execute at the time. Even with memory allocation optimizations, however, we found that many dedicated devices are designed to run a single workflow in the real world. Therefore, it makes no sense to reduce memory usage, and minimized memory usage should not be the primary goal. Instead, we try to maximize the memory usage while such usage doesn't exceed the device's memory size.

3.2 Motivation Example

In embedded systems, scheduling algorithms which follow the precedence constraints are not necessary to find a feasible solution due to memory constraints. We show the example in Fig. 1b. In this graph, each task has the internal buffer and output buffer represented (*Buffer*, *Output*). Considering HEFT, the algorithms we use, the priority list found by the algorithm is [1, 2, 3, 4, 6, 5, 7]. With this sequence, we will first run and allocate the first fourth tasks, [1, 2, 3, 4]. At this moment, if we cannot allocate the memory for task 4, even with *delaying*, the deadlock happens. Since tasks 1, 2, and 3 have to keep their memory usage until task 5 has finished, task 4 cannot be executed until one of them is complete.

4 Problem Modeling

We consider the problem of scheduling dependent tasks on n processors with different computing power, denoted by $p_1, \dots, p_n$. As proposed in previous works [5,

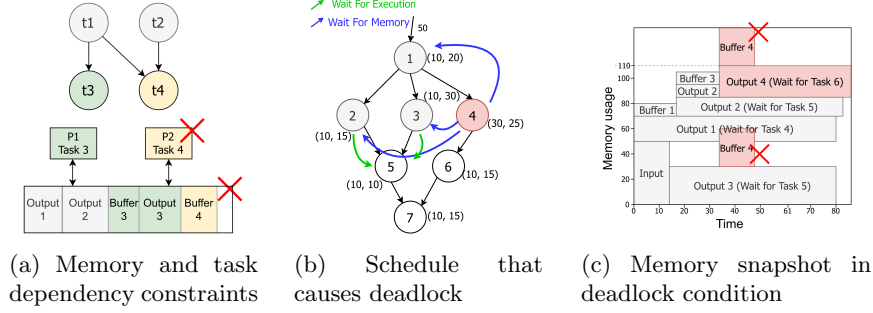


Fig. 1: Illustration of DAG scheduling problem with memory constraint.

8], this type of workflow is often modeled as a directed acyclic graph (DAG). Given a DAG, $G = (V, E)$, where V represents a set of tasks and E represents a set of edges between two adjacent tasks. Each $v \in V$ represents a task and has a weight that represents the buffer size, denoted by mb_v , needed when computing such a task (e.g., weight parameters of a convolution). Each $(i, j) \in E$ represents a precedence constraint that v_i has to complete before v_j can start. In addition, the weight of the edge (i, j) , denoted by mo_i , represents the output produced by task i . In our target environments, the communication cost is neglectable since the processors are either on the same chip or located closely. Our computing environment is described by a $|V| * |P|$ matrix where $|V|$ and $|P|$ represent the number of the tasks, and processors, respectively. For each $v \in V$, it has $|P|$ independent computation cost running on different processors.

4.1 Memory Model

Our target environment includes a shared memory that has a fixed size m , from address 0 to $m - 1$. All memory slots created during the schedule are stored on this single memory. Each slot can be allocated to an arbitrary address but cannot be partitioned, which may introduce fragmentation. In this work, we simply adopt the best-fit strategy and do not further optimize it. We consider the memory constraint as that the total memory usage at any moment in the schedule should be less than m . This usage also considers the fragments, which implies that the actual amount of memory used is less than it.

To run a task v , we need first to allocate two memory slots, internal buffer mb_v and output buffer mo_v occupying a certain amount in the shared memory. mb_v is only used for this task and can be free as soon as this task has finished. On the other hand, mo_v is shared by its direct successors and cannot be free until all its successors are completed. This implies that when this task is running, the input memory mi_p , which is the output memory from its predecessor p , should also be kept in the shared memory. Although this model seems to be simple, this characteristic can affect the schedule results significantly. We use Fig. 1a to illustrate. We can see that even though tasks 1 and 2 have finished the execution, since their output still occupies the memory, tasks 3 and 4 cannot be executed at the same time.

Algorithm 1 List Scheduling with Memory Allocation

Require: $(tasks, M_s)$ **Ensure:** $schedule$

```

1:  $schedule \leftarrow []$ 
2:  $tasks \leftarrow readInput("file")$ 
3:  $calculatePriority(tasks)$ 
4:  $sortedTasks = sortByPriority(tasks)$ 
5: for  $task \in sortedTasks$  do
6:    $est, eft, pid \leftarrow findProcessor(task, schedule)$ 
7:    $ast, aft \leftarrow allocateMemory(task, est, eft)$ 
8:    $freeMemory(task)$ 
9:    $updateTask(pid, ast, aft)$ 
10:   $schedule[pid].append(task)$ 
11: end for
```

Objective The objective function of the problem is to find a schedule that the memory usage at any time is under C_m while its makespan is minimized.

4.2 List Scheduling Definition

In this section, we define the general list scheduling algorithms referred to in this paper as Algorithm 1. First of all, we calculate the priorities for each task in the graph by the function provided by different algorithms, e.g., HEFT calculates *Rank* bottom-up and assigns tasks a higher priority if they have a larger rank, and sort the tasks by the priority in a non-ascending order, $sortedTasks$ in Algorithm 1. For each task in $sortedTasks$, we schedule them on the processor by the function provided by the specified algorithm. For example, HEFT chooses the processor that has EFT(Earliest Finish Time). Note that in this step, we also need to allocate the memory needed for running this task and release the memory when it is done. A schedule fails when there exists a task that cannot allocate the memory.

5 Proposed Solution

In this section, we propose a reservation-based algorithm. Before introducing the solution, we outline certain desired behaviors we want our solution to have. First of all, we know that due to the memory constraint, a schedule may be infeasible due to the ignorance of the memory capacity, leading to the deadlock condition. We will also show that a deadlock could still easily occur even with *delaying* if the sequence isn't changed during the processor selection step. As a result, we hope that our solution will be able to slightly rearrange the sequence. Furthermore, in order to avoid the deadlock, we might potentially decrease the degree of parallelism. Therefore, we want our solution to be adaptable to the degree of parallelism.

List scheduling with memory constraints can be slightly different from that without such limitations. During the execution, tasks might not be able to execute as soon as all of their dependencies have been resolved due to memory constraints. A straightforward solution is to delay its execution until there is enough memory. For this reason, we call this naive solution *delaying*.

First, we discuss why the solution by *delaying* is far from a "good" solution. From our observation, we found that although it follows the ordering from list scheduling algorithms, the effect brought by delaying could be huge. That is, the total delay time is way more significant than the consequence that we reorder the tasks. Furthermore, *delaying* may require a larger memory due to its inflexibility and fixed ordering. We then design our method based on the features mentioned above with the following description. First of all, we should know why the original schedule might be infeasible. The primary goals of most list scheduling algorithms are minimal makespan, which runs the tasks as many as possible if there are any idle processors. For instance, if the tasks in the same layer run simultaneously and occupy all the memory, their successors would never have a chance to be executed. Therefore, we have to reorder the sequence of the task executions. For this purpose, we reserve its successors, which means that we also allocate the memory needed and the schedule slots for its processors, with depth k to ensure that it can be executed with no deadlock and that the memory it uses can be freed since we also guarantee the executions of the successors when looking at a task. The depth k represents that it will reserve for all successors with distance k . The reservation can fail, though. In this case, we will skip to the next task given by the list scheduling algorithm's preordering phase and follow the same steps repeatedly. During this procedure, we achieve the goal of reordering while following the list scheduling algorithm's order as similarly as we can. Though aggressive, this leads to good-quality schedules (the one with a shorter makespan while respecting memory constraints). In addition, we will try different reservation depths k when scheduling a DAG. Since the larger k conceptually leads to lower memory usage and a longer makespan, our method provides an extensive solution set that can fit different use cases.

In Algorithm 2, we first calculate the priority based on different list scheduling algorithms and sort them by their priorities. After that, we iterate through the task in the sorted task list and reserve each of them. We skip the task if it cannot be reserved. In *reserve* function, we have to check if all the predecessors of the task have been allocated yet. If not, we have to allocate them first. Depending on the depth k , we recursively reserve the successors of this task if k does not equal -1. In the function *checkDeps*, we still have to first recursively check the allocation of the dependencies has been done. If it is done, we can now allocate the memory for the current task and schedule it. Here, we use the function *scheduleTask* to represent the code from line 6 to line 10 in Algorithm 1.

It is important to note that we solve potential deadlocks in the processor selection phase, which means that we do not modify the priority list, probably the best sequence for tasks' execution. While respecting memory constraints, we still find schedules with comparably shorter makespan values. Please note

Algorithm 2 Reservation-based heuristic

Require: $(tasks, M_s)$	14: $reservable \leftarrow \text{CHECKDEPS}(task)$
Ensure: $schedule$	15: for $edge \in task.outEdges$ do
1: $schedule \leftarrow []$	16: $reservable \leftarrow reservable \quad \&$
2: $calculatePriority(tasks)$	$\text{RESERVE}(edge.target, k - 1)$
3: $sortedTasks = sortByPriority(tasks)$	17: end for
4: for $task \in sortedTasks$ do	18: end function
5: $reservable \leftarrow \text{RESERVE}(task, k)$	19: function $\text{CHECKDEPS}(task)$
6: if not $reservable$ then	20: $checked \leftarrow \text{True}$
7: $rollback(task)$	21: for $edge \in task.inEdges$ do
8: end if	22: $checked \leftarrow checked \quad \&$
9: end for	$\text{CHECKDEPS}(edge.target)$
10: function $\text{RESERVE}((task, k))$	23: end for
11: if $depth == -1$ then	24: return $checked \quad \&$
12: return True	$scheduleTask(task)$
13: end if	25: end function

that our solution is not deadlock-free. It is possible that our solution cannot find feasible solution. However, we will show the effectiveness of reducing the chance of deadlock in the 6. In addition, we can see that our proposed solution is totally independent of the list scheduling algorithm we choose, and thus it can be easily integrated into existing list scheduling algorithms.

We illustrate our solution with reservation depth $k = 1$ step by step shown in Fig. 2. In the following graphs, the tasks colored with blue represent reserving tasks. The green ones represent the successors of the reserving task with distance k . The red ones represent the tasks that fail to allocate the memory. Lastly, the grey ones represent the dependencies that are needed to be resolved before the reservation. In addition, the bottom of the Fig. 2 represents the memory snapshots of the corresponding reservation step. In these graphs, the blocks are labeled by B or O, representing the internal buffer and output buffer produced by a task.

1. Initially, the entry task, task 1, is selected and reserve the memory for its direct successors in Fig. 2a. When it tries to allocate for task 4, the memory is insufficient, and thus, the reservation fails.
2. During the reservation process, we also need to ensure every dependency is fulfilled. For example, In Fig. 2b, task 1, 3 need to be allocated first.
3. We then have to roll back all tasks allocated by task 1, including itself, i.e., tasks 1, 2, 3, and 4. When task 2 starts its reservation, both memory slots and schedule slots are empty.
4. Then, we find the next task following the priority list and do the same thing again.

After performing our algorithm, we can see that the order of the tasks selecting the processor is actually 1, 2, 3, 5, 4, 6, 7, slightly different from the original priority list. We can also see that in the memory allocation graph, due to the

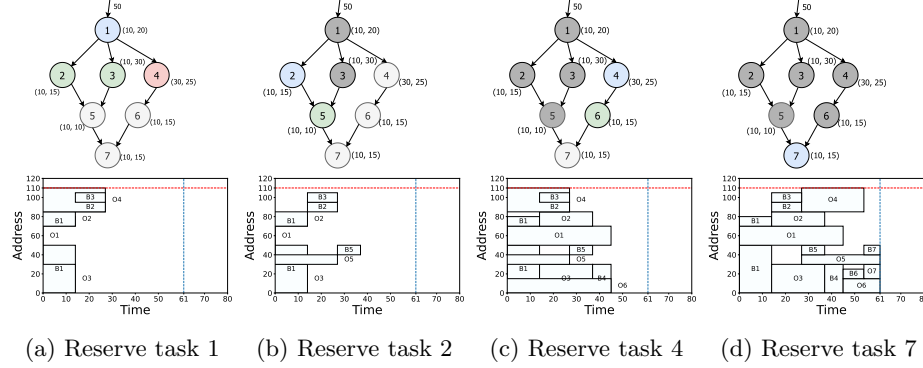


Fig. 2: Scheduling example.

early execution of task 5, we can free the memory occupied by tasks 2 and 3 early. As a result, tasks 4 and 6 can be scheduled afterward.

6 Experimental results

In this section, we evaluated our solution with randomly generated graphs and 5 real world applications. To our best knowledge, SBAC [1] is the latest work that also considers hard memory constraints. We compared our solution with SBAC and showed that our solution outperforms it.

6.1 Random workflows generator.

To evaluate our work, we need a random graph generator. For this purpose, we used the synthetic DAG generation program [7] with three different parameters:

- n : number of DAG nodes. Four different values are used $n = (20, 50, 100, 200)$.
- max_out : maximal number of out degrees of a node. This value leads to different densities of the graphs. $max_out = (1, 2, 3, 4)$.
- α : this parameter affects the height and the width of the DAG. The depth of the DAG is set as $\sqrt{n}/\alpha$. A small value will lead to a thin DAG, whereas a large value induces a fat DAG. The following values are used $\alpha = (0.5, 1.0, 1.5)$.

6.2 Impact of depth k

This section shows the impact of different reservation depth k on the degree of parallelism in Fig 3. We ran our experiment with 100 random graphs and $k = (0, 1, 2, 3, 4, 5)$. Apparently, we can see that when k is larger, the degree of parallelism goes down. The reason behind this is that with a larger k , it may reserve more memory for more predecessors and therefore makes its siblings have less memory to allocate and therefore reduces the degree of parallelism. Meanwhile, the minimum memory decreases along with the depth in the range

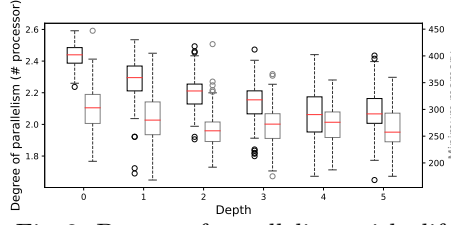
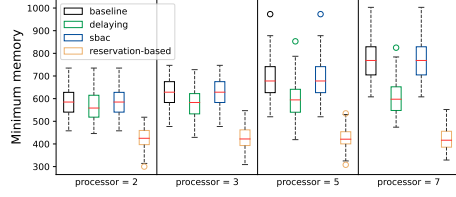
Fig. 3: Degree of parallelism with different k 

Fig. 4: Evaluation of reservation-based algorithm on CPOP

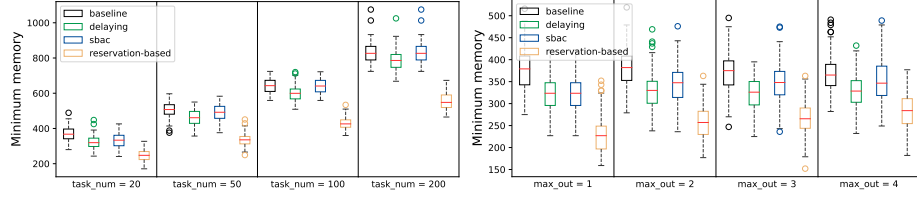


Fig. 5: Minimum memory usage boxplot

of $[0, 3]$ while increasing when the depth is $[3, 5]$. It is because when the depth is too large, the reservation may contain too many tasks where the order of them is fixed and therefore has little impact on improving the minimum memory.

6.3 Minimum memory usage

In this section, we apply our solution to find minimized memory by setting the memory constraints in the range from the usage found by HEFT to the smallest memory usage we can find. The results are shown in Fig. 5. HEFT represents the scheduler that does not consider the memory size at all and works as the baseline. We compared our solution with *delaying* and SBAC.

Randomly generated workflows The randomly generated workflows are controlled by the DAG parameters mentioned above and different processor numbers. We ran each parameter setting with 100 iterations. The results are shown in Fig. 5. We found that except *max_out*, our solution can have a larger improvement when the value of the other parameters increases, as shown in the left figure. We only show the results of *task_num*, for example, due to conserving space. These parameters are directly related to the higher degree of parallelism the schedule could have. Therefore, if an algorithm is too aggressive to optimize for the shorter makespan, it will always try to run as many tasks as possible at the same time, which leads to higher memory usage. On the other hand, our solution takes the memory constraint into account and can effectively minimize memory usage. On the other hand, regarding the *max_out*, our solution has less improvement when it increases, shown in the right figure. The reason behind the result might be the fact that when *max_out* increases, the number of edges will also increase. In this situation, the room for our solution to make an improvement is relatively small.

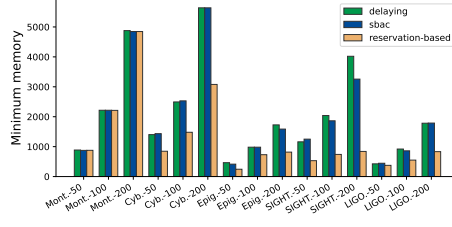


Fig. 6: Evaluation of reservation-based algorithm on real world applications

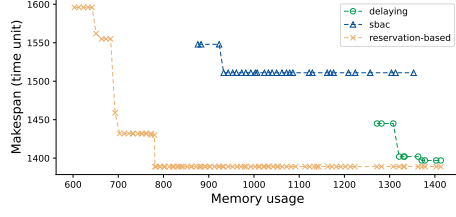


Fig. 7: Trade-off between memory usage and makespan

Real world applications We evaluated our solution on five real world applications: Montage, CyberShake, Epigenomics, SIGHT, and LIGO [4]. We validated our solution with three different graph sizes: 50, 100, and 200. To our best knowledge, there is no such an ML workflow generator for the research purpose; therefore, we chose these workflows to show the generalization of our method. The result is shown in Fig. 6. Apparently, our solution requires much less memory compared with the other two algorithms, except in the case of Montage. One observation is that Montage workflows have a pattern that high parallelism nodes are followed by a bottleneck node(synchronization node). In this case, we have to execute all the nodes in the previous layer before executing such a node. This pattern decreases the effectiveness of the reservation. It is also clear our solutions are way more efficient for large DAGs. This result is similar to the result from the random graphs. A larger DAG provides more opportunities for our solution to make the reservation that can reduce memory usage at some stage of the schedule.

6.4 Minimized makespan value under memory constraints

In this section, we show that the reservation-based algorithm can not only minimize memory usage but also provide better makespan value under certain memory constraints. Besides, the trade-off between the makespan and the memory usage also helps users to determine their desired performance with a certain memory size. In Fig. 7, we can easily see that the points from our method are all equally close or closer to the origin than *delaying* and SBAC. Furthermore, we can see that even with a memory size under around 603, our solution can still find solutions in which *delaying* could only find one with a memory size larger than around 1272 and 876 for SBAC. Furthermore, our solution also has a shorter or equal makespan value when we fix the memory usage.

6.5 Applicability

Since our solution is an extension of list scheduling algorithms, we need to prove that our solution could provide a similar result on algorithms other than HEFT. Therefore, we integrated our solution into Critical-Path-on-a-Processor (CPOP) [11] without modifying its priority and processor selection functions. In Fig. 4, we can see the reservation-based algorithm outperforms the other two

methods, and the improvement is more significant when the number of processors increases.

7 Conclusions

In this paper, we have addressed the scheduling problem under memory constraints and presented a reservation-based algorithm. Our solution prevents deadlock from happening effectively by reordering the sequence in the processor selection phase only when needed. In addition, the experimental results show that our solution can provide a relatively better solution set compared with *delaying* and SBAC in both minimized memory and shorter makespan. Although our approach is mainly designed and evaluated based on the original HEFT, our solution can be generalized to all list-based scheduling.

References

1. Paul-Antoine Arras, Didier Fuin, Emmanuel Jeannot, Arthur Stoutchinin, and Samuel Thibault. List scheduling in embedded systems under memory constraints. *International Journal of Parallel Programming*, 43(6):1103–1128, 2015.
2. Gabriel Bathie, Loris Marchal, Yves Robert, and Samuel Thibault. Revisiting dynamic dag scheduling under memory constraints for shared-memory platforms. In *IEEE IPDPS*, pages 597–606, 2020.
3. Gabriel Bathie, Loris Marchal, Yves Robert, and Samuel Thibault. Dynamic dag scheduling under memory constraints for shared-memory platforms. *International Journal of Networking and Computing*, 11(1):27–49, 2021.
4. Ewa Deelman, Karan Vahi, Mats Rynge, Rajiv Mayani, Rafael Ferreira da Silva, George Papadimitriou, and Miron Livny. The evolution of the pegasus workflow management software. *Computing in Science Engineering*, 21(4):22–36, 2019.
5. Hao Fu, Ce Yu, Jizhou Sun, Mengmeng Wang, and Jun Du. A list scheduling algorithm for dag-based parallel computing models. In *Algorithms and Architectures for Parallel Processing*, pages 406–419, 2015.
6. Wei-Fen Lin, Der-Yu Tsai, Luba Tang, Cheng-Tao Hsieh, Cheng-Yi Chou, Ping-Hao Chang, and Luis Hsu. Onnc: A compilation framework connecting onnx to proprietary deep learning accelerators. In *IEEE AICAS*, pages 214–218, 2019.
7. Livioni. Dag_generator. https://github.com/Livioni/DAG_Generator.git, 2022.
8. Jinhong Luo, Yunfan Zhou, Xijun Li, Mingxuan Yuan, Jianguo Yao, and Jia Zeng. Learning to optimize dag scheduling in heterogeneous environment, 2021.
9. Loris Marchal, Hanna Nagy, Bertrand Simon, and Frédéric Vivien. Parallel scheduling of dags under memory constraints. In *IEEE IPDPS*, pages 204–213, 2018.
10. Narasinga Rao Miniskar, Sirish Kumar Pasupuleti, Vasanthakumar Rajagopal, Ashok Vishnoi, Chandra Kumar Ramasamy, and Raj Narayana Gadde. Optimal sdram buffer allocator for efficient reuse of layer io in cnns inference framework. In *IEEE ISCAS*, pages 1–5, 2018.
11. H. Topcuoglu, S. Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE TPDS*, 13(3):260–274, 2002.